Final Project Option B — Reinforcement Learning Extension

Module M12 | Chapter 14 | Tier: Advanced | Extension worth 35 of 100 final project points

Module: M12 Chapter: 14 Tier: Advanced Included in final project submission

Overview

This option extends the final project by turning part of the simulation model into a sequential decision problem and training an RL agent to control it. It corresponds to **Option B** in `assignments/final_project.tex`.

Choose one controllable decision in your project model and formulate it as a Markov Decision Process (MDP). Examples include staffing level, acceptance or rejection of arrivals, routing, dispatching, or inventory replenishment.

Minimum standard. A strong submission will show that RL is being used for a genuinely sequential decision problem — not merely to rediscover a fixed parameter that could have been found more directly with scenario comparison.

1 Required Components (35 total pts)

1.1 1. MDP Formulation (8 pts)

Formally define:

- **State** s_t: what information is available to the agent at each decision point; justify why it is informative enough for learning.
- **Action** a_t: the set of choices the agent can make; state whether the action space is discrete or continuous.
- **Reward** r_t: the scalar feedback signal; explain the connection between r_t and your project's performance measure (e.g., W_q, cost).
- **Episode boundary**: what constitutes a terminal state or a fixed horizon, and how the environment resets.

1.2 2. Environment Implementation (10 pts)

Implement one of the following:

- A Gymnasium-compatible environment (`gym.Env` subclass) wrapping your SimPy model; or
- A clearly documented environment class with equivalent `reset()`, `step()`, and `render()` semantics.

Your environment must:

1. produce reproducible trajectories given a fixed seed,
2. include at least one sanity check of the observation and action spaces (e.g., assert shapes and bounds), and
3. be importable and runnable from the repo root after `pip install -e simdes/`.

1.3 3. Learning Experiment (7 pts)

Train an agent using one of the following algorithms (or another instructor-approved method):

- Tabular Q-learning (suitable for small discrete state/action spaces)
- DQN (`stable_baselines3.DQN`)
- PPO (`stable_baselines3.PPO`)

Report:

- Algorithm choice and rationale.
- Key hyperparameters: learning rate, discount γ, network architecture (if applicable), total training steps.
- A **learning curve**: episode return vs. training timestep.

1.4 4. Policy Comparison (6 pts)

Compare the learned policy to at least **two baselines**:

1. One simple heuristic (e.g., always open a server when queue exceeds a fixed threshold).
2. One fixed-policy or random baseline.

Present results in a table: policy name, mean return, 95% CI, number of evaluation episodes.

1.5 5. Evaluation and Limitations (4 pts)

- Report mean return over ≥ 30 independent evaluation episodes.
- Explain **one failure mode or limitation** of the learned policy (e.g., exploitation of a reward artifact, poor transfer to a different arrival rate).
- Discuss whether the policy appears to exploit a modelling artifact rather than a genuine structural property of the system.

2 Deliverable

Add a section titled **Reinforcement Learning Extension** to your final project report, following this structure:

1. MDP formulation
2. Environment design and implementation notes
3. Training setup and learning curve
4. Baseline comparison table
5. Results, limitations, and reflection

Include the environment source file (or notebook) in the code repository. The environment must pass `gym.utils.env_checker.check_env()` (or equivalent) without errors.

Grading (within the 35-point extension allocation)

Component	Points
MDP formulation (well-defined, justified)	8
Environment implementation (runnable, reproducible)	10
Learning experiment (curve, hyperparameters)	7
Policy comparison to ≥ 2 baselines	6
Evaluation with stated limitations	4
Total	**35**

Full rubric: `rubrics/final_project_rubric.pdf` | *Exemplar reports: the instructor package (available to instructors on request).*